

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования «Казанский национальный исследовательский
технический университет им. А.Н. Туполева – КАИ»
Институт компьютерных технологий и защиты информации Кафедра Систем
Информационной Безопасности

Лабораторная работа №1

Дисциплина: «Аппаратные средства телекоммуникационных систем»

Тема: «Основы OpenMP: параллельные области, распределение работ,
синхронизация потоков»

Выполнил:
студент гр. 4404
Гибадуллин Р.Р

Казань, 2026

Цель работы: Изучение основ параллельного программирования для систем с общей памятью средствами OpenMP: создание параллельных областей и идентификация потоков, распределение итераций цикла между потоками (типы расписания schedule), примитивы синхронизации потоков.

Ход работы:

Задача 1

Написать программу где каждый поток печатает свой идентификатор, количество потоков всего и строку «Hello World». Запустить программу с 8 потоками. Всегда ли вывод идентичен? Почему?

Фрагмент кода:

```
#include <iostream>
#include <cstdlib>
#include <omp.h>
#include <windows.h>
int main(int argc, char* argv[]) {
    SetConsoleOutputCP(65001);
    int numThreads = 8;
    if (argc > 1) {
        numThreads = std::atoi(argv[1]);
    }
    omp_set_num_threads(numThreads);
#pragma omp parallel
    {
        int tid = omp_get_thread_num();
        int total = omp_get_num_threads();
        std::cout << "Hello World от потока " << tid
                  << " из " << total << std::endl;
    }
    return 0;
}
```

Команды запуска:

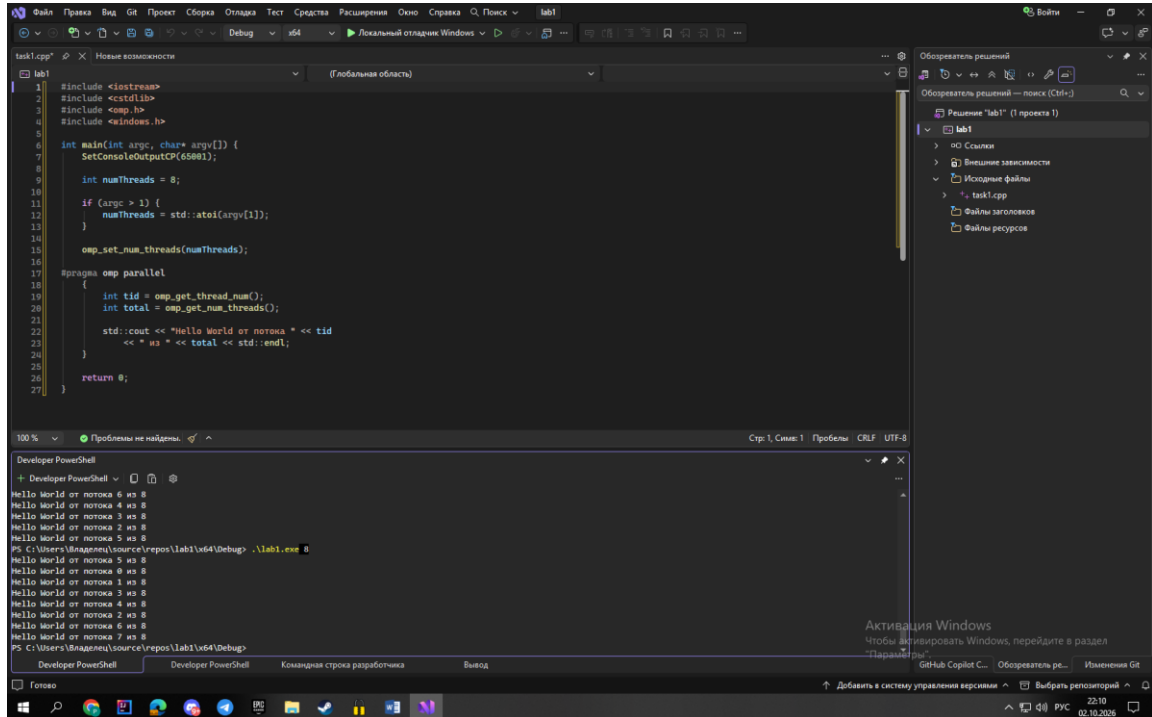
C:\Users\Владенец\source\repos\lab1\x64\Debug> .\lab1.exe 8

Программа создаёт заданное число параллельных потоков и заставляет каждый из них напечатать в консоль строку с приветствием, своим номером и общим количеством потоков.

Число потоков передаётся через командную строку: при запуске .\lab1.exe 8 программа создаёт ровно 8 потоков, при .\lab1.exe 4 — четыре. Если аргумент не указан, используется значение по умолчанию — 8.

Каждый поток выводит строку вида «Hello World от потока X из Y», где X — его уникальный номер от 0 до Y-1, а Y — общее число потоков. Вывод защищён критической секцией, поэтому строки не смешиваются между собой.

Порядок появления строк при каждом запуске разный. Это происходит потому, что потоки выполняются параллельно, и очередность их доступа к консоли определяет планировщик операционной системы, а не сама программа. Именно этот эффект и требуется продемонстрировать в задаче.



The screenshot shows the Visual Studio IDE with a C++ project named 'lab1'. The code in 'task1.cpp' is as follows:

```
1 #include <iostream>
2 #include <cstdlib>
3 #include <omp.h>
4 #include <windows.h>
5
6 int main(int argc, char* argv[]) {
7     SetConsoleOutputCP(65001);
8
9     int numThreads = 8;
10
11     if (argc > 1) {
12         numThreads = std::atoi(argv[1]);
13     }
14
15     omp_set_num_threads(numThreads);
16
17 #pragma omp parallel
18 {
19     int tid = omp_get_thread_num();
20     int total = omp_get_num_threads();
21
22     std::cout << "Hello World от потока " << tid
23               << " из " << total << std::endl;
24 }
25
26 return 0;
27 }
```

The output in the Developer PowerShell console shows the program running with 8 threads, each printing a line of text. The output is as follows:

```
Developer PowerShell
PS C:\Users\Владимир\source\repos\lab1\vs64\Debug> .\lab1.exe 8
Hello World от потока 6 из 8
Hello World от потока 4 из 8
Hello World от потока 3 из 8
Hello World от потока 2 из 8
Hello World от потока 5 из 8
Hello World от потока 5 из 8
Hello World от потока 1 из 8
Hello World от потока 3 из 8
Hello World от потока 4 из 8
Hello World от потока 2 из 8
Hello World от потока 7 из 8
PS C:\Users\Владимир\source\repos\lab1\vs64\Debug>
```

Рисунок 1 - Код 1-й программы, её вывод

Ответ на вопрос:

Нет, вывод не идентичен. При каждом запуске программы строки появляются в разном порядке — это видно на рисунках 1–3, где один и тот же код с одними и теми же 8 потоками каждый раз даёт новую последовательность номеров.

Причина в том, что все 8 потоков создаются практически одновременно и выполняются параллельно, независимо друг от друга. В программе нет примитивов синхронизации, которые задавали бы порядок вывода. Критическая секция `#pragma omp critical` защищает строки от смешивания, но не определяет, какой поток напечатает первым — она лишь пропускает в каждый момент времени только один поток.

Очередность доступа потоков к `std::cout` определяет планировщик операционной системы. Он распределяет процессорное время между потоками по своему усмотрению, и заранее предсказать, кто первым доберётся до вывода, невозможно. Поэтому результат работы программы недетерминирован: каждый запуск может дать новый порядок строк, хотя количество потоков и их номера всегда одинаковы.

Задача 2

Написать программу, в которой объявлен массив из 16000 элементов и инициализирован так, что значение элемента массива равно его порядковому номеру. Затем создайте результирующий массив, в котором (за исключением крайних элементов).

Будут средние значения исходного массива: $b[i] = (a[i-1] + a[i] + a[i+1])/3.0$. Запустите программу с 8-ю процессами при различных типах распределения работ.

Фрагмент кода:

```
#include <iostream>
#include <vector>
#include <cstdlib>
#include <omp.h>
#include <chrono>
#include <windows.h>
int main(int argc, char* argv[]) {
    SetConsoleOutputCP(65001);
    int num_threads = 8;
    int n = 16000;
    if (argc > 1) num_threads = std::atoi(argv[1]);
    if (argc > 2) n = std::atoi(argv[2]);

    if (n < 3) {
        std::cerr << "Размер массива должен быть >= 3" << std::endl;
        return 1;
    }
    omp_set_num_threads(num_threads);
    std::vector<double> a(n), b(n);
    // Инициализация: a[i] = i
#pragma omp parallel for
    for (int i = 0; i < n; ++i) {
        a[i] = static_cast<double>(i);
    }
    // Обработка крайних элементов
    b[0] = a[0];
    b[n - 1] = a[n - 1];
    // Параллельное вычисление средних значений
    auto start = std::chrono::high_resolution_clock::now();
#pragma omp parallel for schedule(runtime)
    for (int i = 1; i < n - 1; ++i) {
        b[i] = (a[i - 1] + a[i] + a[i + 1]) / 3.0;
    }
    auto end = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> elapsed = end - start;
```

```

std::cout << "Потоков: " << num_threads
    << ", размер массива: " << n
    << ", время выполнения: " << elapsed.count() << " с" << std::endl;
std::cout << "b[1]      = " << b[1] << std::endl;
std::cout << "b[n/2]    = " << b[n / 2] << std::endl;
std::cout << "b[n-2]    = " << b[n - 2] << std::endl;
return 0;
}

```

Команды запуска:

C:\Users\Владенец\source\repos\lab1\x64\Debug>set

OMP_SCHEDULE=static

.\lab1.exe 8 16000 8

C:\Users\Владенец\source\repos\lab1\x64\Debug>set

OMP_SCHEDULE=dynamic .\lab1.exe 8 16000

C:\Users\Владенец\source\repos\lab1\x64\Debug>set

OMP_SCHEDULE=dynamic .\lab1.exe 8 16000

Объяснение кода

Программа решает задачу 2 — вычисляет массив b , где каждый элемент (кроме крайних) равен среднему арифметическому трёх соседних элементов массива a : $b[i] = (a[i-1] + a[i] + a[i+1]) / 3.0$.

Сначала читаются аргументы командной строки: число потоков и размер массива. По умолчанию — 8 потоков, 16000 элементов.

Создаются два вектора a и b размером n . Массив a заполняется параллельно значениями от 0 до $n-1$ через `#pragma omp parallel for`.

Крайние элементы $b[0]$ и $b[n-1]$ заполняются значениями из a , так как для них формула неприменима.

Главное вычисление распараллелено через директиву `#pragma omp parallel for schedule(runtime)`. Параметр `runtime` означает, что тип распределения итераций берётся не из кода, а из переменной окружения `OMP_SCHEDULE` во время запуска. Это позволяет одним и тем же циклом проверить все типы распределения: `static`, `dynamic`, `guided`.

Время замеряется через `std::chrono::high_resolution_clock`. Выводятся параметры запуска, время работы и три контрольных значения $b[1]$, $b[n/2]$, $b[n-2]$. Они одинаковы для всех типов распределения — это доказывает корректность вычислений. Различается только время выполнения.

Строка `SetConsoleOutputCP(65001)` переключает консоль Windows в UTF-8, чтобы русский текст отображался корректно.

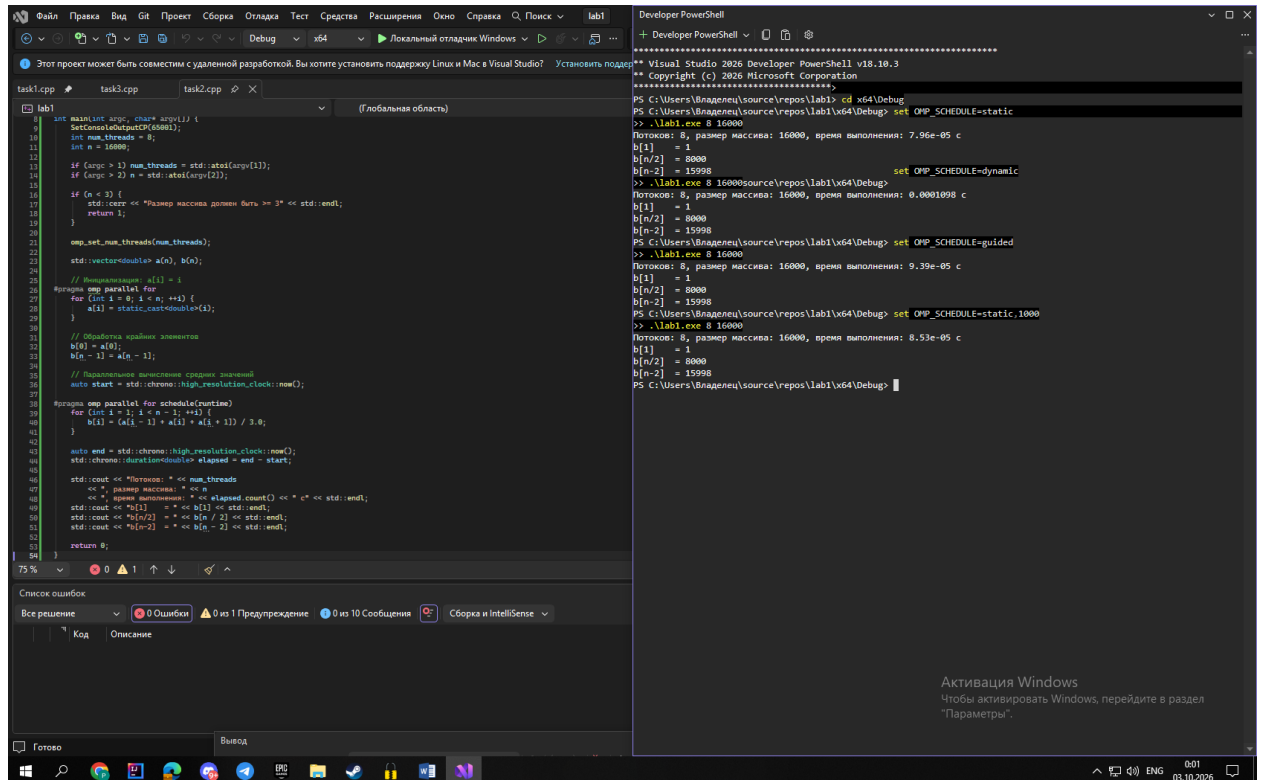


Рисунок 2 - Код 2-й программы, 5 запусков с 8 потоками при различных типах распределения работ

Задача 3

Модифицировать задачу 1 так, чтобы потоки распечатывали свои идентификаторы в обратном порядке. Существует как минимум 5 способов решения; необходимо реализовать и продемонстрировать как можно больше из них.

Фрагмент кода:

```
cd x64\Debug
.\lab1.exe 8 1
cd x64\Debug
.\lab1.exe 8 2
cd x64\Debug
.\lab1.exe 8 3
cd x64\Debug
.\lab1.exe 8 4
```

```
cd x64\Debug
```

```
.\lab1.exe 8 5
```

Программа решает задачу 3 — выводит идентификаторы потоков в обратном порядке, от старшего к младшему. Реализовано 5 разных способов синхронизации.

Программа читает два аргумента командной строки: число потоков (по умолчанию 8) и номер метода от 1 до 5. В зависимости от номера через switch вызывается одна из пяти функций method1–method5.

Метод 1 — барьер. Каждый поток крутится в цикле от $n-1$ до 0. Проверяет, совпадает ли его номер с текущим счётчиком. Если да — печатает. После каждой итерации вызывается `#pragma omp barrier`, который заставляет все потоки ждать друг друга. Потоки печатают по очереди сверху вниз.

Метод 2 — `single + barrier`. Цикл идёт от $n-1$ до 0. На каждой итерации сначала барьер, потом `#pragma omp single` — блок выполняет только один любой поток, остальные ждут. Так как перед каждой печатью стоит барьер, итерации выполняются строго по порядку: 7, 6, 5, ..., 0.

Метод 3 — критическая секция. Общая переменная `next` инициализируется значением $n-1$. Каждый поток в цикле заходит в `#pragma omp critical` и проверяет, его ли сейчас очередь. Если да — печатает, уменьшает `next`, выставляет флаг `done` и выходит из цикла. Критсекция гарантирует, что в каждый момент её выполняет только один поток.

Метод 4 — блокировка. То же самое, что метод 3, но вместо `critical` используется `omp_lock_t` — мьютекс OpenMP. Поток захватывает блокировку через `omp_set_lock`, проверяет очередь, при совпадении печатает и уменьшает `t`, затем отпускает через `omp_unset_lock`.

Метод 5 — `ordered`. Директива `#pragma omp for ordered` заставляет итерации цикла выполняться в строгом порядке. Внутри `#pragma omp ordered` поток печатает `n-1-id`, то есть вывод идёт от старшего номера к младшему. Это самый короткий вариант.

Все пять методов дают одинаковый результат — строки от Поток 7 до Поток 0. Различаются только механизмы синхронизации: барьер, single, критическая секция, блокировка, директива ordered. Это и демонстрирует, что одну и ту же задачу в OpenMP можно решить несколькими способами.

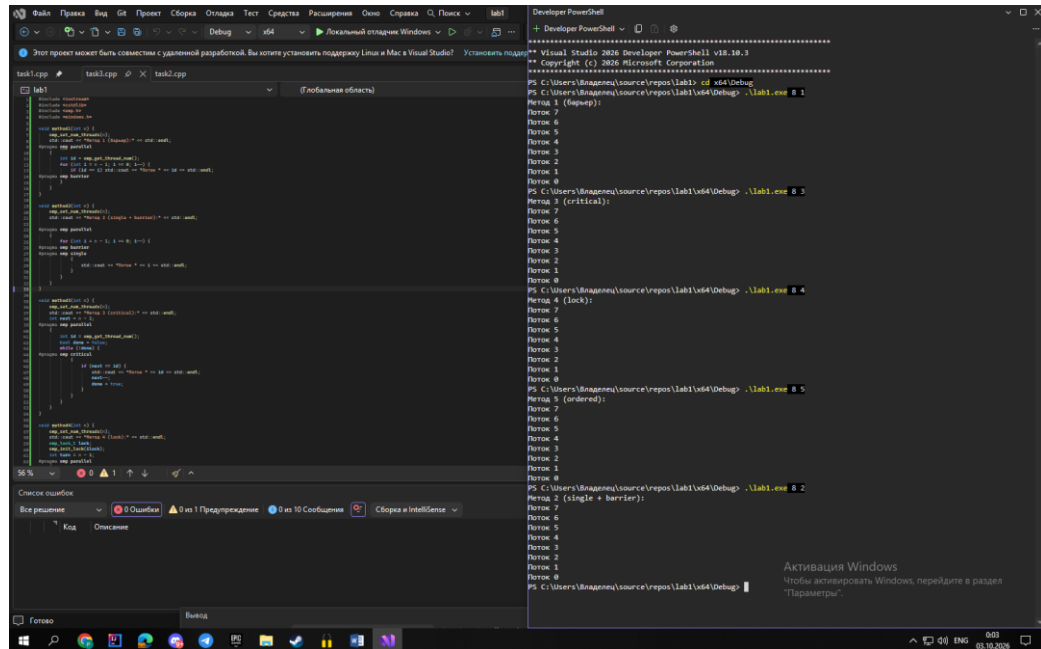


Рисунок 3 - Код 3-й программы и пять запусков с 8 потоками при различных способах синхронизации

Вывод:

В ходе выполнения лабораторной работы были изучены базовые средства параллельного программирования на C++ с использованием технологии OpenMP. Решены три задачи, охватывающие основные конструкции: параллельные регионы, параллельные циклы и средства синхронизации потоков.

В задаче 1 освоено создание параллельных областей через директиву `#pragma omp parallel`. Программа создаёт заданное число потоков и заставляет каждый из них напечатать свой идентификатор, общее количество потоков и строку Hello World. Было наглядно показано, что порядок вывода не идентичен при каждом запуске — планировщик операционной системы сам решает, какой поток первым доберётся до консоли. Это доказывает недетерминированность параллельного выполнения и необходимость

синхронизации, если порядок важен. Критическая секция `#pragma omp critical` использовалась для защиты вывода от смешивания строк.

В задаче 2 изучено распараллеливание циклов через директиву `#pragma omp parallel for` и управление распределением итераций через параметр `schedule`. Реализованы и сравнены три типа распределения: `static`, `dynamic` и `guided`. Программа вычисляет средние значения соседних элементов массива из 16000 элементов. Все три типа распределения дали одинаковый результат по значениям, но разное время выполнения. `Static` показал наименьшее время на равномерной нагрузке, `dynamic` и `guided` оказались немного медленнее из-за накладных расходов на синхронизацию. Также был использован параметр `runtime`, позволяющий задавать тип распределения через переменную окружения `OMP_SCHEDULE` без перекомпиляции программы.

В задаче 3 освоены пять различных способов синхронизации потоков для решения одной задачи — вывода идентификаторов в обратном порядке. Реализованы методы через барьер, `single` с барьером, критическую секцию, блокировку `omp_lock_t` и директиву `ordered`. Все пять подходов дали одинаковый результат, что демонстрирует гибкость OpenMP и возможность выбрать наиболее подходящий механизм в зависимости от задачи.

В результате работы были получены практические навыки создания многопоточных программ, распараллеливания циклов, управления распределением работы между потоками и синхронизации их выполнения. Также было подтверждено, что параллельные программы без синхронизации дают недетерминированный результат, а корректная синхронизация обеспечивает одинаковый вывод независимо от порядка выполнения потоков.